Universidad de Buenos Aires

Facultad de Ciencias Exactas y Naturales

TP2: AirMiles

Grupo JavaConMates

Integrantes:

Ayala Ignacio
Della Bonzana Lara
Morales Santiago
Vannelli Tiziana

25 de octubre de 2026

1. Introducción

En este documento se justifican las estructuras de datos seleccionadas para implementar el TAD `AirMiles`. Para cada estructura se describe la información que almacena, las operaciones que permite realizar y su relación con las complejidades temporales esperadas.

Denotamos por U la cantidad de usuarios registrados, por P la cantidad de promociones y por $|L|$ la longitud del nombre de una promoción. Además, N representa la cantidad de usuarios solicitados para el ranking.

El diseño busca que las operaciones de consulta y modificación accedan directamente a la información que necesitan, evitando recorridos innecesarios. Las complejidades que dependen del ABB se analizan distinguiendo el caso promedio del peor caso, ya que el árbol utilizado no realiza balanceo automático.

2. Almacenamiento e identificación de usuarios

2.1. Estructura elegida

Los usuarios se almacenan en un árbol binario de búsqueda (ABB). Cada nodo contiene un objeto `Usuario`, y el orden del árbol se determina por el identificador único (`id`).

La clase `Usuario` implementa `Comparable<Usuario>`. Su método `compareTo` compara los identificadores mediante `Integer.compare`. Esto permite que el ABB localice un usuario a partir de su ID sin confundirlo con otro usuario que tenga distintos valores de millas.

2.2. Justificación

El ABB permite organizar los usuarios por identificador y realizar búsquedas e inserciones siguiendo un único camino desde la raíz. En un árbol razonablemente equilibrado, estas operaciones requieren tiempo $O(\log U)$.

2.3. Operaciones asociadas

El ABB se utiliza para:

- `registrarUsuario` y `registrarUsuarioConReferidor`: insertar usuarios.
- `consultarSaldo` y `consultarCategoria`: localizar al usuario por ID.
- `acumularMillas`: encontrar la cuenta que se debe actualizar.
- `canjearMillas`: localizar al usuario que realiza el canje.
- `transferirMillas`: localizar al usuario de origen y al de destino.
- `paresTransferidoresExclusivos`: permitiría recorrer todos los usuarios, si la implementación de esta operación lo requiere.

La operación auxiliar `buscarUsuario` centraliza el acceso al ABB y evita repetir la lógica de búsqueda en los distintos métodos de `AirMiles`.

3. Representación de cada usuario

Cada usuario se representa mediante un objeto `Usuario`. El objeto agrupa los datos de su cuenta, por lo que, una vez localizado en el ABB, las operaciones pueden consultar o modificar sus atributos directamente.

Entre los atributos considerados se encuentran:

- `id`: identificador del usuario.
- `millasDisponibles`: saldo que puede utilizar.
- `millasTotales`: millas acumuladas históricamente.
- `millasCanjeadas`: millas utilizadas en canjes.
- `millasTransferidas`: millas transferidas a otros usuarios.
- `posicionHeap`: posición actual del usuario en el max-heap.

Los atributos que representan cantidades históricas permiten actualizar las estadísticas sin tener que reconstruirlas a partir de todas las operaciones anteriores. Por su parte, `posicionHeap` permite localizar al usuario dentro del heap en tiempo $O(1)$ antes de actualizar su prioridad.

La comparación de usuarios por ID se mantiene independiente de su saldo: el ABB necesita un orden estable basado en la identidad del usuario, mientras que el heap utiliza el saldo disponible para determinar las prioridades.

4. Promociones

4.1. Estructura elegida

Las promociones se almacenan en un trie o árbol de prefijos. Cada nodo contiene un arreglo de 26 hijos, uno por cada letra del alfabeto, y puede almacenar la información de una promoción cuando representa el final de su nombre.

La información de cada promoción se agrupa en un objeto `Promocion`, que contiene:

- `factor`: multiplicador de la promoción.
- `tope`: cantidad máxima de millas que se pueden otorgar utilizando la promoción, cuando tiene límite.
- `millasOtorgadas`: cantidad de millas que ya consumieron el tope.
- `tieneTope`: indica si existe un límite.

La promoción inicial `NombrePromo`, con factor 1, se agrega al crear el sistema.

4.2. Justificación

El trie permite localizar una promoción recorriendo los caracteres de su nombre. Si el acceso a los hijos de cada nodo cuesta $O(1)$, buscar, insertar y eliminar una promoción requiere $O(|L|)$, independientemente de la cantidad total de promociones almacenadas.

Esta estructura es adecuada porque las operaciones sobre promociones dependen de sus nombres. Además, agrupar los datos de cada promoción en un mismo objeto permite con-

sultar el factor y controlar el tope sin mantener estructuras separadas para cada atributo.

4.3. Control del tope

Para acumular millas se calcula primero la cantidad base, aplicando el factor de categoría del usuario. Luego, el método `calcularMillas` busca la promoción una sola vez y calcula cuántas millas se obtendrían al aplicar su factor.

Si la promoción no tiene tope, se aplica el factor normalmente. Si tiene tope, se verifica que la cantidad acumulada más las millas de la nueva operación no supere el límite. Cuando la operación es válida, se actualiza `millasOtorgadas`. Si superaría el tope, se otorgan únicamente las millas base y el acumulado de la promoción no se modifica.

Por lo tanto, el tope cuenta todas las millas otorgadas en las acumulaciones en las que se aplica la promoción, no solo las millas adicionales producidas por el multiplicador.

Este control requiere una única búsqueda en el trie, por lo que la complejidad de esta parte es $O(|L|)$.

La implementación supone que los nombres de promoción están formados por letras de la A a la Z, sin espacios ni otros caracteres. Si la especificación admite un alfabeto más amplio, la representación de los hijos debe adaptarse.

5. Máximos históricos

Las operaciones `elMasAcumulador` y `elMasCanjeador` deben consultar usuarios que alcanzan determinados máximos históricos.

Para evitar recorrer el ABB en cada consulta, se mantienen referencias directas a los usuarios que ocupan esas posiciones: `maxAcumulador` y `maxCanjeador`.

Cuando un usuario acumula millas, se actualiza su contador histórico y se compara con el máximo registrado. De manera análoga, cuando un usuario canjea o transfiere millas, se actualiza la información que corresponda al criterio definido por la especificación para el máximo canjeador.

Si el usuario supera el máximo actual, basta con reemplazar la referencia correspondiente. La consulta posterior consiste en acceder a esa referencia y devolver el ID, por lo que cuesta $O(1)$.

Al registrar el primer usuario, las referencias deben inicializarse para evitar consultar un máximo que todavía sea `null`. También debe respetarse el criterio de desempate que establezca la especificación.

Esta estrategia evita recorrer todos los usuarios en cada consulta, a cambio de mantener las referencias actualizadas durante las operaciones que modifican los contadores.

6. Ranking de millas disponibles

6.1. Estructura elegida

Para obtener los usuarios con mayor saldo disponible se utiliza un max-heap. La prioridad de cada elemento se determina por `millasDisponibles`; el usuario con mayor saldo se encuentra en la raíz.

El heap es genérico y recibe las operaciones necesarias para comparar elementos y consultar o actualizar sus posiciones. De esta forma, puede funcionar con objetos `Usuario` sin depender de la implementación de las cuentas.

6.2. Justificación

La propiedad de max-heap garantiza que la prioridad de cada padre es mayor o igual que la de sus hijos. No exige que el hijo izquierdo sea mayor que el derecho ni mantiene a todos los elementos ordenados.

Esta propiedad permite consultar el máximo en $O(1)$ y extraerlo en $O(\log U)$. Para obtener los primeros N usuarios, se extrae repetidamente el máximo. Como las extracciones modifican temporalmente el heap, los elementos extraídos se guardan y luego se vuelven a insertar, de modo que el ranking no altera el conjunto de usuarios registrados.

Tanto las extracciones como las reinserciones cuestan $O(\log U)$ cada una. Por eso, obtener y restaurar los primeros N elementos cuesta $O(N \log U)$, suponiendo $N \leq U$.

6.3. Actualización de prioridades

Cuando cambia el saldo disponible de un usuario, su prioridad en el heap también puede cambiar. Para actualizarla eficientemente, cada usuario conserva su posición actual mediante `posicionHeap`.

Así se evita buscar el usuario recorriendo todo el heap. Una vez conocida su posición, se restaura la propiedad de max-heap moviendo el elemento hacia arriba o hacia abajo según corresponda, en tiempo $O(\log U)$.

La posición se actualiza cada vez que se intercambian elementos. Cuando un usuario se extrae, su posición se marca como inválida; al reinsertarlo, se establece nuevamente.

La complejidad $O(N \log U)$ supone que el heap contiene a todos los usuarios y que se mantiene actualizado después de cada operación que modifica el saldo.

7. Transferencias entre usuarios

La operación `transferirMillas` localiza al usuario de origen y al de destino mediante el ABB. Luego modifica los saldos disponibles de ambas cuentas y actualiza los contadores históricos que correspondan.

Si el ranking se mantiene en un max-heap, ambas prioridades deben actualizarse después

de modificar los saldos. Cada búsqueda en el ABB cuesta $O(\log U)$ en promedio y $O(U)$ en el peor caso; cada actualización del heap cuesta $O(\log U)$.

Para identificar pares de transferidores exclusivos no basta con conocer los totales transferidos por cada usuario. Es necesario conservar información sobre las relaciones entre origen y destino, de modo que se pueda distinguir entre transferencias recíprocas, transferencias hacia terceros y repeticiones entre una misma pareja.

La estructura auxiliar elegida para estas relaciones debe permitir verificar la exclusividad y evitar contar dos veces un mismo par. La implementación de `paresTransferidoresExclusivos` debe completar este mantenimiento y demostrar que la consulta cumple la complejidad exigida por la especificación.

8. Complejidades temporales

La siguiente tabla resume las complejidades derivadas de las estructuras descriptas. Las operaciones que consultan o modifican usuarios dependen de la altura del ABB; por eso se indica tanto el caso promedio como el peor caso.

Operación	**Complejidad temporal**
Crear el sistema	$O(1)$ para inicializar las estructuras
Registrar usuario	$O(\log U)$ promedio; $O(U)$ peor caso
Buscar usuario	$O(\log U)$ promedio; $O(U)$ peor caso
Acumular millas	$O(\log U + \|L\|)$ promedio; $O(U + \|L\|)$ peor caso
Canjear millas	$O(\log U)$ promedio; $O(U)$ peor caso
Transferir millas	$O(\log U)$ promedio; $O(U)$ peor caso
Consultar saldo o categoría	$O(\log U)$ promedio; $O(U)$ peor caso
Iniciar o finalizar promoción	$O(\|L\|)$
Controlar tope de promoción	$O(\|L\|)$
Consultar máximo histórico	$O(1)$
Consultar máximo del heap	$O(1)$
Actualizar prioridad en el heap	$O(\log U)$
Obtener y restaurar los primeros N del ranking	$O(N \log U)$
Pares de transferidores exclusivos	Depende de completar la estructura auxiliar

Las complejidades de las operaciones que utilizan el ABB suponen que las implementaciones de sus métodos básicos cumplen las cotas indicadas. Como el árbol no se balancea automáticamente, no se garantiza $O(\log U)$ en el peor caso.

Las complejidades de las operaciones del heap suponen que las comparaciones y las consultas o actualizaciones de posición cuestan $O(1)$.

9. Conclusión

Las estructuras elegidas responden a necesidades diferentes del TAD. El ABB organiza los usuarios por identificador y permite localizar sus cuentas; el trie permite administrar promociones en función de la longitud de sus nombres; y el max-heap permite obtener los usuarios con mayor saldo sin ordenar todas las cuentas en cada consulta.

Las referencias a los máximos históricos permiten responder esas consultas en tiempo constante, mientras que la posición almacenada por cada usuario facilita mantener el heap actualizado cuando cambian los saldos.

La corrección del sistema depende de mantener consistentes estas estructuras auxiliares durante las operaciones. En particular, deben actualizarse los saldos, los contadores históricos, las prioridades del heap y el consumo de los topes promocionales. La representación de las relaciones de transferencia debe completarse de acuerdo con la condición de exclusividad establecida por la especificación.